INTELIGÊNCIA ARTIFICIAL APLICADA

Instituto NTA

FASE 1 · ORQUESTRAÇÃO COM ESTADO

Aula 1.3

Workflows II: paralelização e orchestrator-workers

Quando você sabe quais são as subtarefas, paralelize. Quando só o modelo sabe quantas são, orquestre.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção
Projeto: Núcleo de Agentes Operacionais
Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código
Pré-requisito: Aula 1.2 (tag **aula-1.2**)

Prof. Thiago Azeredo Rodrigues · Série pública · 2026

1. Objetivo de aprendizagem

Ao final desta aula você terá implementado paralelização e orchestrator-workers em Python puro e como grafo, e saberá reconhecer, diante de um problema concreto, se as subtarefas são conhecidas de antemão ou se é o modelo que precisa decidir quantas são.

2. Por que esta aula importa

A aula anterior terminou com um caso que ela não resolvia. Uma mensagem como esta:

> **O CASO QUE QUEBRA OS PADRÕES ANTERIORES**
>
> "*Oi, aqui é a Ana da TechFlow. Três coisas: o relatório de exportação está dando erro 500 desde ontem e isso está travando o meu time; a gente quer entender o plano Pro porque vamos crescer para 40 usuários no trimestre; e preciso remarcar a call de quinta.*"

O chaining não serve, porque o caminho dele é fixo e essa mensagem pede três caminhos. O router não serve, porque ele escolhe **uma** rota e aqui as três estão certas. Não é falta de habilidade na modelagem: é limite de expressão. Nenhuma topologia desenhada de antemão resolve um problema cujo número de partes só se conhece depois de ler a entrada.

E esse caso não é artificial. É o e-mail que chega todo dia na caixa de suporte de qualquer SaaS, e é exatamente onde a maior parte dos assistentes em produção responde uma coisa e ignora as outras duas.

Esta aula resolve isso, e no caminho resolve também um problema mais comum e mais barato: tarefas independentes que estão esperando umas pelas outras sem motivo.

3. Teoria

3.1 Dois padrões que parecem um só

Paralelização e orchestrator-workers são tratados juntos em toda a literatura, e isso confunde. Eles resolvem problemas diferentes, e a diferença é a aula inteira:

> **A DISTINÇÃO QUE ORGANIZA A AULA**
>
> **Paralelização: você** sabe quais são as subtarefas. Elas estão escritas no seu código. Elas só não precisam esperar umas pelas outras.
> **Orchestrator-workers:** o **modelo** decide quais e quantas subtarefas existem. Você não sabe antes de rodar.

Do primeiro você ganha **latência**. Do segundo você ganha **capacidade**: ele resolve o que nenhum dos padrões anteriores conseguia expressar.

3.2 Paralelização, e o que ela não economiza

Três perguntas independentes sobre a mesma mensagem do cliente: qual é o tom, há urgência real, há oportunidade comercial. Nenhuma depende do resultado da outra. Rodá-las em sequência é esperar três vezes por nada.

Aqui cabe o aviso que falta na maior parte do material sobre o assunto, porque é contra-intuitivo: **paralelizar economiza tempo, não dinheiro**. Três chamadas continuam sendo três chamadas, e você paga pelas três. O que muda é esperar dois segundos em vez de seis. Quem paraleliza esperando a conta cair vai se decepcionar, e vai descobrir isso na fatura.

Uma dúvida razoável aparece nesse ponto: se são três perguntas sobre o mesmo texto, por que não juntar tudo num prompt só, que seria mais barato? Porque seria pior. Vale aqui a regra da Aula 1.2: instruções que competem entre si produzem respostas piores. Três instruções estreitas acertam mais do que uma que tenta fazer três coisas. Você paga três chamadas e recebe três respostas melhores.

3.3 Por que threads bastam

Chamada de API é trabalho de **I/O**: a thread fica bloqueada esperando a rede, e nesse intervalo o Python libera o interpretador para as outras. É exatamente o caso em que threads funcionam bem, apesar do GIL.

Se o trabalho fosse de CPU, isso não valeria, e seria preciso processo em vez de thread. Saber distinguir os dois casos é o que evita tanto o paralelismo que não acelera nada quanto a complexidade de multiprocessing onde uma thread resolvia.

3.4 Orchestrator-workers, e por que ele precisa existir

Volte à ponte da aula anterior: uma mensagem que traz três problemas diferentes. Erro em produção, interesse no plano Pro, remarcar uma reunião. Tudo num parágrafo.

O chaining não serve: o caminho dele é fixo e essa mensagem pede três caminhos. O router não serve: ele escolhe **uma** rota, e aqui as três estão certas. Nenhuma topologia que você consiga desenhar de antemão resolve isso, porque o número de partes não é conhecido antes de ler a mensagem.

O orquestrador resolve em três etapas. **Decompor:** o modelo lê a tarefa e devolve uma lista de subtarefas, e é ele quem decide o tamanho dessa lista. **Delegar:** cada subtarefa vai para um worker. **Sintetizar:** os resultados voltam e viram uma resposta só.

> **ANALOGIA**
> O router é a recepção do escritório: identifica o seu problema e te manda para a sala certa. O orquestrador é o advogado que lê o seu caso, decide que ele tem três frentes, distribui cada uma para um colega e depois escreve *um* parecer juntando as três. A recepção não decide quantas frentes existem. O orquestrador decide.

3.5 A síntese não é enfeite

É tentador tratar a síntese como um passo cosmético e devolver os três resultados concatenados. Não faça. Sem ela o cliente recebe três respostas desconexas sobre o próprio e-mail, e a experiência fica pior do que se você tivesse respondido uma coisa só e bem.

A síntese é o passo que transforma N resultados num texto que parece escrito por uma pessoa. Num orquestrador, ela é a parte que o cliente lê.

3.6 Fan-out, fan-in e o limite da aresta comum

No grafo, várias arestas saindo do mesmo nó se chamam **fan-out**, e o reencontro depois se chama **fan-in**. Declarar paralelismo passa a ser desenhar a topologia: o LangGraph roda no mesmo passo tudo que está pronto para rodar, e só executa o nó de reencontro quando todos terminarem. Aquela espera, que no Python puro foi o `.result()` de cada futuro, sai de graça.

Mas o fan-out estático não resolve o orquestrador, e aqui está o limite: **toda aresta que você desenha liga nó a nó, e é decidida na montagem**. Não existe como desenhar de antemão um grafo com N nós quando N é decidido em tempo de execução.

3.7 A Send API, que é a peça nova

A resposta do LangGraph para isso é a `Send`. Uma função de aresta condicional, em vez de devolver o *nome* de um nó, devolve uma **lista** de `Send`, e cada `Send` é uma instância do nó de destino com a sua própria entrada.

Três subtarefas viram três execuções paralelas de **um único nó declarado**. Cinco subtarefas viram cinco. O grafo continua estático no desenho e vira dinâmico na execução, e é só por causa dessa peça que o orquestrador cabe num grafo.

Há um detalhe de projeto embutido nela que vale notar: cada `Send` carrega o estado daquele worker, não o estado inteiro do grafo. Cada worker recebe só o seu pedaço. É a mesma regra de contexto menor da aula anterior, agora aplicada ao fan-out.

3.8 O reducer, e o bug que ele evita

Quando N nós escrevem na mesma chave do estado, alguém precisa dizer como juntar. Isso é o **reducer**, que a Aula 1.1 já introduziu e que aqui deixa de ser detalhe e passa a ser obrigatório.

Sem reducer, a escrita padrão é substituir, e o último nó a terminar sobrescreve os outros. O resultado desse bug é traiçoeiro porque **parece plausível**: você recebe uma análise em vez de três, ela está correta, e nada estoura. Só falta coisa. Bug de fan-out é quase sempre assim, e é por isso que ele merece teste próprio.

3.9 Como escolher, na prática

Padrão	Use quando	O que você ganha
Chaining	As etapas são fixas, sequenciais e você as conhece	Previsibilidade e qualidade por passo
Routing	Os tipos de pedido são distintos e só um se aplica	Prompt especializado por tipo
Paralelização	Você conhece as subtarefas e elas são independentes	Latência, e nada mais
Orquestrador	O número de subtarefas não é conhecido antes de rodar	Capacidade de expressar o problema
Agente	A tarefa é aberta e o número de passos também	Autonomia, pelo preço dela

A regra da Fase 1 continua valendo, e o orquestrador é a hora de aplicá-la com rigor: **comece pelo padrão mais simples que resolve o problema**. O orquestrador é caro, são duas

chamadas de modelo só de overhead, uma para decompor e uma para sintetizar, antes de qualquer trabalho útil. Se um router resolve, use o router.

4. Mão na massa

Dois arquivos novos: `nucleo/paralelo.py` (Python puro) e `nucleo/grafo_paralelo.py` (os mesmos dois padrões como grafo). E, pela primeira vez na Fase 1, uma **correção** em arquivo da Fase 0: o `observabilidade.py`. O paralelismo expôs um problema que não tinha como aparecer enquanto tudo era sequencial, e esse conserto é um dos pontos altos da aula.

Passo 1 — As três análises, que são fixas

O que e por quê: cada análise é uma chamada de modelo com uma pergunta estreita. Elas são escritas por você, e é justamente isso que separa este padrão do orquestrador do segundo bloco.

```
def _analisar(nome, pergunta, mensagem, provider):
    """Uma analise isolada sobre a mesma mensagem.

    Cada uma e uma chamada de modelo com uma pergunta estreita. Juntar
    as tres num prompt so seria mais barato, e pior: instrucoes que
    competem entre si produzem respostas piores (regra da Aula 1.2).
    """
    with observar(nome, tipo="generation", entrada=mensagem):
        return chat(provider,
                    [{"role": "user", "content": mensagem}],
                    system=pergunta)


# As tres perguntas sao FIXAS: nos as escrevemos. E essa a marca do
# padrao, e o que o separa do orquestrador logo abaixo.
ANALISES = {
    "tom": ("Classifique o tom da mensagem do cliente em uma palavra: "
            "calmo, impaciente ou irritado. Responda so a palavra."),
    "urgencia": ("A mensagem indica urgencia real de negocio? Responda "
                 "em uma frase curta, comecando com sim ou nao."),
    "oportunidade": ("Ha oportunidade comercial nesta mensagem (upgrade, "
                     "expansao, nova contratacao)? Responda em uma frase "
                     "curta, comecando com sim ou nao."),
}
```

Repare que `_analisar` abre um span do tipo `generation`: a observabilidade da Aula 0.7 continua valendo, e vamos cobrá-la daqui a dois passos.

Passo 2 — Rodar as três ao mesmo tempo

O que e por quê: `ThreadPoolExecutor` basta porque o trabalho é de I/O. O `.result()` bloqueia até aquele futuro terminar, mas os três já estão rodando desde o `submit`: o tempo total é o da análise mais lenta, não a soma das três.

```
def analisar_em_paralelo(mensagem, provider="anthropic", analises=None):
    """Roda as analises ao mesmo tempo e devolve {nome: resultado}.

    ThreadPoolExecutor e suficiente aqui porque chamada de API e
    I/O bound: a thread fica bloqueada esperando a rede, e o Python
    libera o interpretador nesse intervalo. Para trabalho de CPU isso
    nao valeria, e seria preciso processo em vez de thread.

    Repare no em_thread() envolvendo cada chamada. Sem ele o codigo
```

```
    roda igual e o TRACE fica vazio, porque thread nova nao herda
    contexto. E um bug silencioso: o resultado esta certo e a
    observabilidade mente.
    """
    analises = analises or ANALISES

    with iniciar_trace("paralelizacao", entrada=mensagem):
        with ThreadPoolExecutor(max_workers=len(analises)) as executor:
            futuros = {
                nome: executor.submit(
                    em_thread(_analisar, nome, pergunta, mensagem,
                              provider))
                for nome, pergunta in analises.items()
            }
            # .result() bloqueia ate aquele futuro terminar, mas os
            # tres ja estao rodando desde o submit. O tempo total e o
            # da analise mais lenta, nao a soma das tres.
            return {nome: f.result() for nome, f in futuros.items()}
```

Resultado esperado ao rodar: um dicionário com as três chaves preenchidas, em aproximadamente o tempo de uma chamada só.

Passo 3 — O bug que o paralelismo revelou

O que e por quê: escreva o teste que confere se os passos paralelos aparecem no trace, e ele falha. Não falha por pouco: o trace vem **vazio** na parte paralela. Vale parar aqui, porque esse é o tipo de defeito que chega em produção sem ninguém perceber.

A causa está na Aula 0.7. O tracer guardava a pilha de passos abertos num `ContextVar`, e isso funcionou perfeitamente por oito aulas. Só que uma **thread nova não herda contexto nenhum**: dentro dela, o `ContextVar` volta ao valor padrão, todo `observar` encontra um trace inexistente e vira no-op. O resultado continua correto. Só a observabilidade mente.

> **E TEM UM SEGUNDO PROBLEMA, PIOR**
>
> Mesmo forçando o contexto a atravessar, a pilha era uma **lista mutável**. Um `ContextVar` isola a *referência*, não o objeto apontado: duas threads leriam a mesma lista e dariam `append` e `pop` nela ao mesmo tempo, e o pai de cada passo viraria sorteio. A correção certa é guardar um **valor imutável**, o passo atual, e deixar cada contexto fazer o seu próprio `set`.

O `observar` e o `iniciar_trace` passam a ficar assim:

```
# O PASSO CORRENTE. ContextVar em vez de variavel global para o dia em
# que o Nucleo rodar com varias requisicoes simultaneas (Fase 4): cada
# contexto enxerga o seu proprio trace.
#
# ATENCAO ao que esta aqui, porque mudou na Aula 1.3. Ate a Aula 1.2
# esta variavel guardava uma LISTA (uma pilha de passos abertos), e
# funcionou perfeitamente enquanto tudo era sequencial. O paralelismo
# expos o problema: um ContextVar isola a REFERENCIA por contexto, nao
# o objeto apontado. Duas threads lendo o mesmo ContextVar recebiam A
# MESMA LISTA e davam append e pop nela ao mesmo tempo, e o pai de cada
# passo virava sorteio.
#
# A correcao e guardar um VALOR IMUTAVEL: o passo atual. Cada contexto
# faz o seu proprio .set(), que nenhum outro ve. A regra geral vale
# fora daqui: ContextVar com objeto mutavel dentro nao isola nada.
_atual: ContextVar = ContextVar("passo_atual", default=None)
```

```
@contextmanager
def observar(nome, tipo="span", entrada=None, modelo=""):
    """Abre um passo e o fecha ao sair do bloco.

    O aninhamento sai de graca: cada passo aberto vira filho do passo
    que estiver corrente. E isso que forma a arvore.
    """
    pai = _atual.get()
    if pai is None:
        # Ninguem abriu um trace: nao rastreamos, mas o codigo roda.
        yield None
        return

    passo = Passo(nome=nome, tipo=tipo, entrada=entrada,
                  modelo=modelo, inicio=time.time())
    # list.append e atomico sob o GIL, entao varios filhos paralelos
    # chegam inteiros. A ORDEM entre eles nao e garantida, e nenhum
    # codigo deve depender dela.
    pai.filhos.append(passo)
    token = _atual.set(passo)

    # A ponte com a plataforma externa, quando existir, abre o mesmo
    # passo la. Nada do resto do Nucleo precisa saber disso.
    externo = _abrir_externo(nome, tipo, entrada)
    try:
        yield passo
    finally:
        passo.fim = time.time()
        _atual.reset(token)
        _fechar_externo(externo, passo)


@contextmanager
def iniciar_trace(nome, entrada=None):
    """Abre a execucao inteira. Tudo que acontecer dentro vira filho."""
    raiz = Passo(nome=nome, tipo="trace", entrada=entrada,
                 inicio=time.time())
    token = _atual.set(raiz)
    externo = _abrir_externo(nome, "trace", entrada)
    try:
        yield raiz
    finally:
        raiz.fim = time.time()
        _atual.reset(token)
        _fechar_externo(externo, raiz)
        _enviar_pendentes()
```

E entra uma função nova para o caso da thread crua. Ela devolve um *callable* em vez de executar, e isso é proposital: o `copy_context()` precisa acontecer na thread de origem, no momento do `submit`.

```
def em_thread(funcao, *args, **kwargs):
    """Embala uma chamada para rodar em outra thread SEM perder o trace.

    O problema, em uma frase: uma thread nova nao herda contexto
    nenhum. Dentro dela, _atual.get() devolve None, todo observar()
    vira no-op e o trace fica com um buraco exatamente onde o trabalho
    paralelo aconteceu.

    A correcao e copiar o contexto de quem SUBMETE e rodar a funcao
    dentro dessa copia. Por isso esta funcao devolve um callable em vez
    de executar: o copy_context() precisa acontecer na thread de
```

```
    origem, no momento do submit.

        executor.submit(em_thread(minha_funcao, arg1, arg2))

    E exatamente o que o LangGraph faz por conta propria nos nos
    paralelos dele, e e por isso que a versao em grafo desta aula nao
    precisa chamar isto.
    """
    ctx = copy_context()
    return lambda: ctx.run(funcao, *args, **kwargs)
```

No `analisar_em_paralelo`, cada `submit` passa a receber `em_thread(...)`, como o código do Passo 2 já mostra. Com isso o teste passa.

> **A LIÇÃO, QUE VALE ALÉM DESTA AULA**
>
> Instrumentação que não é testada não é instrumentação: é decoração. O bug aqui não tinha sintoma no resultado, só no trace, e só apareceu porque existia um teste olhando a árvore. Em produção, você descobriria num incidente, procurando o passo que sumiu.

Passo 4 — O contrato do plano

O que e por quê: começa o segundo padrão. O modelo vai decidir quantas subtarefas existem, e um modelo decidindo sem cerca inventa tipos que não existem. O `Literal` é a cerca, pela regra da Aula 0.5: contrato primeiro.

```
class Subtarefa(BaseModel):
    """Uma parte do pedido, definida pelo proprio modelo."""

    tipo: Literal["comercial", "suporte", "agenda"] = Field(
        description="comercial: plano, preco, upgrade. suporte: erro ou "
                    "falha tecnica. agenda: marcar, remarcar ou cancelar "
                    "reuniao")
    pedido: str = Field(description="O que precisa ser resolvido, em "
                                    "uma frase")


class Plano(BaseModel):
    """O resultado da decomposicao: quantas partes e quais."""

    subtarefas: list[Subtarefa] = Field(
        description="Uma entrada por pedido distinto na mensagem. Se a "
                    "mensagem traz um pedido so, devolva uma entrada. "
                    "Nao invente pedidos que nao estao no texto")
```

Olhe a descrição do campo `subtarefas` com atenção. Ela diz explicitamente para devolver uma entrada quando houver um pedido só, e para não inventar pedidos que não estão no texto. Sem essas duas frases o orquestrador infla: ele decompõe um "oi, tudo bem?" em quatro tarefas.

Passo 5 — Decompor, que é a etapa de orquestração

O que e por quê: este passo só planeja, não resolve nada. Reaproveita o `extrair_estruturado` da Aula 0.6 inteiro, inclusive o ciclo de correção quando o modelo erra o formato.

```
def decompor(mensagem, provider="anthropic"):
    """A etapa de ORQUESTRACAO: so planeja, nao resolve nada.

    E aqui que o padrao se diferencia: o numero de subtarefas sai do
```

```
    modelo, nao do codigo. Duas mensagens diferentes geram planos de
    tamanhos diferentes.
    """
    with observar("decompor", tipo="generation", entrada=mensagem):
        return extrair_estruturado(mensagem, Plano, provider=provider)
```

Passo 6 — Os workers, e por que eles seguram o próprio erro

O que e por quê: a resiliência da Aula 0.6 reaparece, e num fan-out ela pesa mais do que em qualquer outro padrão: as outras partes podem ter dado certo, e jogar tudo fora por causa de uma seria desperdício. Erro de um worker vira observação, não morte do conjunto.

```
def _rodar_worker(subtarefa, workers):
    """Executa um worker. Erro de um nao derruba os outros."""
    with observar(f"worker:{subtarefa.tipo}",
                  entrada=subtarefa.pedido) as sp:
        funcao = workers.get(subtarefa.tipo)
        if funcao is None:
            saida = f"[sem worker para &#x27;{subtarefa.tipo}']"
        else:
            try:
                saida = funcao(subtarefa)
            except Exception as e:
                # Mesma regra da Aula 0.6: falha de uma parte vira
                # informacao, nao morte do conjunto. Num fan-out isso
                # pesa mais ainda, porque as outras partes podem ter
                # dado certo.
                saida = f"[falhou: {type(e).__name__}: {e}]"
        if sp is not None:
            sp.saida = saida
        return saida
```

Repare que um tipo sem worker registrado também não estoura. Isso é diferente do router da Aula 1.2, que levantava `ValueError`, e a diferença é proposital: no router a rota é uma e sem ela não há resposta; num fan-out, as outras partes seguem.

Passo 7 — Sintetizar

O que e por quê: juntar N resultados numa resposta só. É o passo que o cliente lê.

```
def sintetizar(mensagem, resultados, provider="anthropic"):
    """A etapa de SINTESE: junta as partes numa resposta so.

    Sem ela o cliente recebe tres respostas desconexas. O sintetizador
    e o que transforma N resultados num texto que parece escrito por
    uma pessoa.
    """
    partes = "\n".join(f"- {tipo}: {saida}" for tipo, saida in resultados)
    sistema = ("Voce e o assistente do FlowDesk, um SaaS B2B. Junte os "
               "resultados abaixo numa resposta unica, curta e cordial "
               "ao cliente, em portugues. Nao repita o que ja foi dito "
               "e nao invente nada que nao esteja nos resultados.")
    conteudo = f"Mensagem original: {mensagem}\n\nResultados:\n{partes}"

    with observar("sintetizar", tipo="generation", entrada=conteudo):
        return chat(provider, [{"role": "user", "content": conteudo}],
                    system=sistema)
```

Passo 8 — O orquestrador completo

O que e por quê: decompor, delegar e sintetizar. E os dois padrões da aula se combinam aqui: a decomposição é dinâmica, mas as partes continuam independentes entre si, então os workers também rodam em paralelo.

```
def orquestrar(mensagem, workers, provider="anthropic", paralelo=True):
    """Decompoe, delega e sintetiza.

    workers  -> dict {"comercial": fn, "suporte": fn, "agenda": fn}
    paralelo -> roda os workers ao mesmo tempo (padrao) ou em sequencia

    Devolve (resposta, plano, resultados).
    """
    with iniciar_trace("orquestrador", entrada=mensagem) as raiz:
        plano = decompor(mensagem, provider)

        if not plano.subtarefas:
            return ("Nao identifiquei um pedido claro na mensagem. "
                    "Pode detalhar?"), plano, []

        # Os workers tambem rodam em paralelo: a decomposicao e
        # dinamica, mas as partes continuam independentes entre si.
        # Os dois padroes desta aula se combinam.
        if paralelo and len(plano.subtarefas) > 1:
            with ThreadPoolExecutor(
                    max_workers=len(plano.subtarefas)) as ex:
                futuros = [ex.submit(em_thread(_rodar_worker, s, workers))
                           for s in plano.subtarefas]
                saidas = [f.result() for f in futuros]
        else:
            saidas = [_rodar_worker(s, workers) for s in plano.subtarefas]

        resultados = [(s.tipo, saida)
                      for s, saida in zip(plano.subtarefas, saidas)]
        resposta = sintetizar(mensagem, resultados, provider)

        if raiz is not None:
            raiz.saida = resposta
        return resposta, plano, resultados
```

Resultado esperado: com a mensagem de três pedidos, três subtarefas, três resultados e uma resposta única. Com uma mensagem de um pedido, o mesmo código roda um worker. Nenhuma linha dele decidiu esse número.

Passo 9 — Fan-out estático no grafo

O que e por quê: no Python puro, paralelizar foi trabalho seu. No grafo é declarativo: várias arestas saindo de `START` e nada mais. Note o `operator.or_` como reducer, que é o merge de dicionários; sem ele o último nó a terminar sobrescreveria os outros.

```
class EstadoAnalise(TypedDict):
    mensagem: str
    provider: str
    analises: Annotated[dict, operator.or_]
    # operator.or_ e o merge de dicionarios do Python (o mesmo do
    # operador |). Cada no devolve a SUA chave, e o reducer junta as
    # tres. Sem ele, o ultimo no a terminar sobrescreveria os outros, e
    # esse bug e dificil de ver porque o resultado parece plausivel:
    # so falta coisa.
```

```
def construir_analise(analises=None, checkpointer=None):
    """Fan-out estatico: as analises sao conhecidas na montagem."""
    analises = analises or ANALISES

    def _fazer_no(nome, pergunta):
        def no(estado: EstadoAnalise) -> dict:
            saida = _analisar(nome, pergunta, estado["mensagem"],
                              estado["provider"])
            return {"analises": {nome: saida}}   # so a propria chave
        return no

    g = StateGraph(EstadoAnalise)
    for nome, pergunta in analises.items():
        g.add_node(nome, _fazer_no(nome, pergunta))
        # Varias arestas saindo de START: e isto, e so isto, que
        # declara o paralelismo. O LangGraph roda no mesmo super-step
        # tudo que esta pronto para rodar.
        g.add_edge(START, nome)
        g.add_edge(nome, END)

    return g.compile(checkpointer=checkpointer)


def rodar_analise(mensagem, provider="anthropic", grafo=None,
                  analises=None):
    """Roda as analises em paralelo pelo grafo."""
    if grafo is None:
        grafo = construir_analise(analises)

    with iniciar_trace("paralelizacao_grafo", entrada=mensagem):
        final = grafo.invoke({"mensagem": mensagem, "provider": provider,
                              "analises": {}})
    return final["analises"]
```

UMA COISA CONCRETA QUE O FRAMEWORK RESOLVE

Esta versão não chama `em_thread` em lugar nenhum, e o trace sai completo. O LangGraph copia o contexto para os nós paralelos por conta própria. Tem um teste na aula medindo exatamente isso, e é um argumento honesto a favor do grafo que não aparece em comparação nenhuma: não é só topologia.

Passo 10 — Fan-out dinâmico, com a Send API

O que e por quê: aqui entra a peça que não existia até agora. A função de aresta condicional não devolve o nome de um nó: devolve uma lista de `Send`, uma por subtarefa que o modelo definiu. Três subtarefas viram três execuções paralelas de um único nó declarado.

```
class EstadoOrquestra(TypedDict):
    mensagem: str
    provider: str
    subtarefas: Optional[list]
    resultados: Annotated[list, operator.add]
    resposta: str


class EstadoWorker(TypedDict):
    """O estado que UM worker recebe.

    Repare que e diferente do estado do grafo. Isso e proposital e e
    um recurso do Send: cada worker recebe so o seu pedaco, em vez do
```

```
    estado inteiro. Contexto menor por worker, pela mesma regra da
    Aula 1.2.
    """
    subtarefa: dict
    provider: str


def construir_orquestrador(workers, checkpointer=None):
    """Monta o orquestrador. O numero de workers e decidido em runtime."""

    def planejar(estado: EstadoOrquestra) -> dict:
        plano = decompor(estado["mensagem"], estado["provider"])
        # Guardamos dicts, nao objetos: a regra do estado serializavel
        # da Aula 1.1 continua valendo.
        return {"subtarefas": [s.model_dump() for s in plano.subtarefas]}

    def distribuir(estado: EstadoOrquestra):
        """A funcao de fan-out dinamico.

        Ela nao devolve o NOME de um no, como as condicionais que voce
        ja escreveu. Devolve uma LISTA de Send, e cada Send e uma
        instancia do no worker com a sua propria entrada.

        Tres subtarefas viram tres execucoes paralelas de um unico no
        declarado. E assim que um grafo estatico atende a uma
        decomposicao dinamica.
        """
        if not estado["subtarefas"]:
            return "sintetizar"           # nada a delegar
        return [Send("worker", {"subtarefa": s,
                                "provider": estado["provider"]})
                for s in estado["subtarefas"]]

    def worker(estado: EstadoWorker) -> dict:
        subtarefa = Subtarefa(**estado["subtarefa"])
        saida = _rodar_worker(subtarefa, workers)
        # O reducer acumula os resultados dos N workers numa lista so.
        return {"resultados": [(subtarefa.tipo, saida)]}

    def sintetizar_no(estado: EstadoOrquestra) -> dict:
        if not estado["resultados"]:
            return {"resposta": ("Nao identifiquei um pedido claro na "
                                 "mensagem. Pode detalhar?")}
        with observar("sintetizar_no"):
            texto = sintetizar(estado["mensagem"], estado["resultados"],
                               estado["provider"])
        return {"resposta": texto}

    g = StateGraph(EstadoOrquestra)
    g.add_node("planejar", planejar)
    g.add_node("worker", worker)
    g.add_node("sintetizar", sintetizar_no)

    g.add_edge(START, "planejar")
    g.add_conditional_edges("planejar", distribuir,
                            ["worker", "sintetizar"])
    # O FAN-IN: todo worker desagua no sintetizador, e o LangGraph so
    # roda o sintetizador quando TODOS terminarem. Essa espera e de
    # graca; no Python puro ela foi o .result() de cada futuro.
    g.add_edge("worker", "sintetizar")
    g.add_edge("sintetizar", END)

    return g.compile(checkpointer=checkpointer)
```

Três detalhes merecem atenção. O `EstadoWorker` é **diferente** do estado do grafo, porque cada `Send` carrega só o pedaço daquele worker. As subtarefas são guardadas como `dict`, pela regra de estado serializável da Aula 1.1. E a aresta `worker → sintetizar` é o fan-in: o LangGraph só roda o sintetizador quando todos os workers terminarem, e essa espera sai de graça.

Passo 11 — Os testes

O que e por quê: o que interessa provar aqui não é o texto que o modelo devolve, é o comportamento da topologia. Em `tests/test_paralelo.py`, doze testes, nenhuma chamada de API. Três merecem destaque.

O primeiro é o único teste de tempo do curso, e ele existe porque o ganho do padrão é temporal: não há como provar paralelismo olhando só o resultado, que é idêntico ao da versão sequencial. A margem é generosa de propósito, porque teste de tempo em CI compartilhada falha por ruído, não por regressão.

```
def test_paralelizacao_e_mais_rapida_que_a_soma(monkeypatch):
    """Tres chamadas de 200ms levam ~200ms, nao 600ms.

    Este e o unico teste do curso que mede TEMPO, e ele existe porque
    o ganho do padrao e temporal: nao da para provar paralelismo
    olhando so o resultado, que e identico ao da versao sequencial.

    A margem e generosa (400ms) de proposito: teste de tempo em CI
    compartilhada falha por ruido, nao por regressao.
    """
    def lento(*a, **k):
        time.sleep(0.2)
        return "ok"

    monkeypatch.setattr(paralelo, "chat", lento)

    inicio = time.perf_counter()
    saida = paralelo.analisar_em_paralelo("mensagem qualquer")
    gasto = time.perf_counter() - inicio

    assert len(saida) == 3
    assert gasto < 0.4          # sequencial seria >= 0.6
    # Repare no que NAO melhorou: continuam sendo tres chamadas, e voce
    # paga pelas tres. Paralelizar troca latencia, nao custo.
```

O segundo prova que o número de workers vem do modelo, e não do código: o mesmo teste roda com um plano de três e um plano de um.

```
@pytest.mark.parametrize("plano,esperado", [(PLANO_TRES, 3), (PLANO_UM, 1)])
def test_quantidade_de_workers_segue_o_plano(monkeypatch, plano, esperado):
    """A marca do padrao: o mesmo codigo roda 1 ou 3 workers.

    Nenhuma linha do nosso codigo decide esse numero. Ele sai do
    modelo. E por isso que nao da para desenhar este grafo de
    antemao, e e por isso que a Send API existe.
    """
    _modelo(monkeypatch, plano)
    _, _, resultados = paralelo.orquestrar("mensagem", WORKERS)
    assert len(resultados) == esperado

    _modelo(monkeypatch, plano)
    _, res_grafo = grafo_paralelo.rodar_orquestrador("mensagem", WORKERS)
    assert len(res_grafo) == esperado
```

O terceiro é a regressão do tracer. Sem o `em_thread` ele falha; com ele, passa. Bug silencioso só não volta se tiver teste.

```
def test_span_dentro_de_thread_entra_na_arvore():
    """O bug que esta aula descobriu, virado teste.

    Ate a Aula 1.2 o tracer guardava uma pilha mutavel num ContextVar.
    Funcionou enquanto tudo era sequencial. Numa thread, o observar()
    virava no-op e o trace ficava com um buraco: resultado certo,
    observabilidade mentindo.

    Sem o em_thread(), este teste falha. Com ele, passa. E e por isso
    que ele existe: bug silencioso so nao volta se tiver teste.
    """
    from concurrent.futures import ThreadPoolExecutor

    from nucleo.observabilidade import em_thread, iniciar_trace, observar

    def trabalho(nome):
        with observar(nome) as sp:
            return sp is not None

    with iniciar_trace("fan_out") as raiz:
        with ThreadPoolExecutor(2) as ex:
            futuros = [ex.submit(em_thread(trabalho, f"t{i}"))
                       for i in range(2)]
            criados = [f.result() for f in futuros]

    assert all(criados)
    assert sorted(f.nome for f in raiz.filhos) == ["t0", "t1"]
    # Os dois sao filhos da RAIZ, nao um filho do outro. Com a pilha
    # compartilhada da versao antiga, o aninhamento virava sorteio.
```

Passo 12 — Rodar com API real e medir

O que e por quê: o exemplo em `exemplos/orquestrar_pedido.py` roda os dois padrões com a mensagem de três pedidos e, principalmente, **mede**: ele faz as três análises em sequência e depois em paralelo, e imprime os dois tempos lado a lado. Sem o contrafactual, o ganho é fé.

```
$ python exemplos/orquestrar_pedido.py

sequencial: 5.84s
paralelo:   2.11s
ganho:      2.8x

O modelo decompos em 3 subtarefas:
  - suporte    erro 500 na exportacao de relatorio travando o time
  - comercial  quer entender o plano Pro para 40 usuarios
  - agenda     remarcar a call de quinta
```

Os números variam a cada execução, e o ganho nunca chega a 3x porque há overhead de thread e a latência das três chamadas não é igual. O que não varia é a ordem de grandeza.

5. Portabilidade

Fan-out e fan-in não são conceitos de IA. É o `scatter-gather` de sistemas distribuídos, o `map` seguido de `reduce`, o `Promise.all` do JavaScript, o grupo de goroutines com `WaitGroup` em Go.

Você já fez isso antes, com outro nome. O que muda é que cada ramo agora é uma chamada de modelo, e portanto tem custo e latência altos o suficiente para a paralelização valer a pena.

O orquestrador também tem nome fora daqui: é um `task queue` com um planejador na frente. A diferença é que o planejador é o modelo, e é essa diferença que exige o contrato Pydantic na saída dele.

E a `Send`, apesar do nome específico, resolve um problema universal: instanciar dinamicamente um passo declarado estaticamente. O CrewAI faz isso com tarefas geradas em runtime, o AutoGen com agentes criados por um orquestrador, e um sistema caseiro faria com uma fila e workers consumindo dela. **O conceito é portável; a sintaxe não.**

O que esta aula ensinou sobre `ContextVar` é portável para qualquer linguagem com contexto por thread: o contexto não atravessa fronteira de thread sozinho, e estado mutável dentro dele não isola nada. Vale para o `context` do Go e para o `AsyncLocalStorage` do Node tanto quanto para o Python.

6. Terminei quando

- As três análises rodam em paralelo e o tempo total é próximo do de **uma** chamada, não da soma das três. Você mediu isso, não supôs.
- O trace da execução paralela mostra os três passos. Se mostrar o trace vazio, o `em_thread` não está no lugar.
- Você manda a mensagem com três pedidos e o orquestrador devolve **três** subtarefas que ele mesmo definiu, com uma resposta única no fim.
- Você manda uma mensagem com um pedido só e o mesmo código roda **um** worker, sem você mudar nada.
- Um worker que levanta exceção não derruba os outros: a resposta sai com as partes que deram certo.
- A versão em grafo e a versão em Python puro produzem o mesmo resultado, e o teste prova.
- Você sabe explicar, sem olhar a anotação, por que o orquestrador precisa da `Send` e a Aula 1.2 não precisava.

7. Erros comuns e troubleshooting

- **Paralelizei e a conta subiu.** Era o esperado. Paralelização troca latência por nada: o custo é o mesmo da versão sequencial. Se a expectativa era economizar, o padrão escolhido estava errado desde o começo.
- **Paralelizei e não acelerou.** Duas causas prováveis. Ou o trabalho não é de I/O, e aí thread não ajuda, ou você está medindo a primeira chamada, que carrega cliente e conexão. Meça com a sequencial do lado, como o exemplo da aula faz.
- **Chegou só uma análise em vez de três.** Reducer faltando ou errado na chave do estado. Sem ele a escrita padrão substitui, e o resultado parece certo com menos itens. Teste que conta os itens pega isso; teste que olha o conteúdo não.
- **O trace ficou vazio exatamente na parte paralela.** O `ContextVar` não atravessa thread. Uma thread nova começa sem contexto, e todo `observar` dentro dela vira no-op. É o bug mais

silencioso da aula, porque o resultado continua certo e só a observabilidade mente. A correção é copiar o contexto de quem submete.

- **Os passos paralelos aparecem aninhados um no outro.** Estado mutável dentro de `ContextVar`. Um `ContextVar` isola a referência, não o objeto apontado: duas threads recebem a mesma lista e o pai de cada passo vira sorteio. Guarde valor imutável.
- **O modelo inventou um tipo de subtarefa que não existe.** Falta `Literal` no campo do contrato. Sem a cerca, o fan-out manda trabalho para workers inexistentes.
- **Um worker falhou e derrubou tudo.** Exceção não contida dentro do worker. Num fan-out isso pesa mais do que em qualquer outro padrão, porque as outras partes podem ter dado certo e você joga fora.
- **O orquestrador decompôs uma mensagem simples em cinco partes.** A descrição do campo da lista está permissiva demais. Diga explicitamente que uma mensagem com um pedido só devolve uma entrada e que ele não deve inventar pedidos que não estão no texto.

CÓDIGO COMPLETO DESTA AULA

Repositório da série, com o projeto no estado exato desta aula:

`github.com/azrosolucoestecnologicas/engenharia-ia-aplicada` · tag `aula-1.3`

`git clone` e depois `git checkout aula-1.3`

8. Ponte para a próxima aula

O orquestrador decompõe, delega e sintetiza. Mas ele não **julga** o que produziu: a resposta sintetizada sai como saiu, boa ou ruim, e ninguém confere.

Na **Aula 1.4, evaluator-optimizer**, entra o último padrão de workflow: um segundo modelo avalia a saída do primeiro contra critérios explícitos e devolve para correção, em ciclo, até passar. É o padrão que transforma qualidade em algo mensurável em vez de opinião, e é a última peça antes de a Fase 1 virar autonomia de verdade.